3) Insertion Sort:-

→
```
#include<iostream>
#include<algorithm>
using namespace std;
void insertion(int a[], int n){
    for(int i=1; i<n; i++){
        int j=i;
        while(j>0 && a[j]<a[j-1]){
            swap(a[j], a[j-1]);
            j--;
        }
    }
}
int main(){
    int a[]={5, 2, 4, 6, 1, 3};
    int n=6;
    insertion(a, n);
    for(int i=0; i<n; i++){
        cout<<a[i]<<" ";
    }
}
```

* O/P:→

1 2 3 4 5 6

4) Intersection:-

→
```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
int main(){
    int a[100], b[100], s1, s2;
    vector<int> v;
    cout<<"Enter size of arr 1 and 2";
    cin>>s1>>s2;
    cout<<"Enter arr 1:";
    for(int i=0; i<s1; i++){
        cin>>a[i];
    } cout<<"Enter arr 2:";
    for(int i=0; i<s2; i++){
        cin>>b[i];
    for(int i=0; i<s1; i++){
        for(int j=0; j<s2; j++){
            if(a[i]==b[j] && find(v.begin(), v.end(), a[i])==v.end()){
                v.push_back(a[i]);
                break; } } }
    cout<<"Intersection:";
    for(int i=0; i<v.size(); i++){
        cout<<v[i]; }
    return 0;
}
```

* O/P:- Enter size of arry 1 and 2: 3
Enter arr 1: 1 2 3
Enter arr 2: 2 3 4
Intersection: 23

24/4/26

Experiment No 8

i) Palindrome:-

→
```
#include <iostream>
#include <algorithm>
using namespace std;
int main(){
    string s;
    cin>>s;
    string rev = s;
    reverse (rev.begin(), rev.end());

    if (s == rev) cout<<"Palindrome";
    else cout<<"Not palindrome";
}
```

* O/P:→

```
101
Palindrome.
```

ii) Substring:-

→
```
#include <iostream>
using namespace std;
int main() {
    string s, sub;
    cin >> s >> sub;
    int pos = s.find(sub);
    if (pos != -1)
    cout << pos;

    else
        cout <<-1;
}
```

* O/P:→

```
substring
string
1
```

iii) Anagram:-

→
```
#include<iostream>
#include<algorithm>
using namespace std;
int main(){
    string a,b;
    cin >> a >>b;

    sort(a.begin(), a.end());
    sort(b.begin(), b.end());

    if(a==b)
    cout<<" Anagram.";
    else
    cout << " Not Anagram";
}
```

* O/P:→

Parth
Rathp
Anagram

iv) Leftmost repeating character:-

→
```
#include<iostream>
using namespace std;
int main() {
    string s;
    cin>> s;

    for(int i = 0;i<s.length(); itt) {
        for(int j = i+1 ; j<s.length(); jtt){
            if(s[i] == s[j]){
                cout<<s[i];
            }
    }
}
    cout <<-1;
}
```

* O/P:→

parthp
p

v) reverse words

```
#include<iostream>
using namespace std;

int main(){
    string s;
    getline(cin,s);

    string word ="";
    string result = "";

    for(int i =s.length()-1; i>=0;i++) {
        if(s[i] ==' '){
            result += word + " ";
            word = "";
        }else{
            word = s[i] +word;
        }
    }
    result += word;
    cout << result;
}
```

* O/P:→

```
parth is 17
17 is parth
```

24/4/26

Experiment NO. 9

* 1) Stack using array:-

```
#include<iostream>
using namespace std;
int #define MAX 5
int stack[MAX] top = -1;
void push (int x) {
        if (top ==MAX-1) cout <<" overflow";
        else
            stack[++top] = x;
        }
void pop {
        if(top == -1) cout << Underflow;"
        else
            cout << stack[top--] <<"popped";
        }
void display(){
        for (int i = top; i >= 0; i--) {
            cout<< stack[i] <<" ";
            cout << endl;
}}
int main(){ push (10);
            push (20);
            push (30);
            display();
            pop();
            display();
}
```

P.T.O.

* O/P: →
30 20 10
30 popped
20 10
=> Dry Run: →
Start: top = -1
push(10) → top=0 → [10]
push(20) → top=1 → [10, 20]
push(30) → top=2 → [10, 20, 30]
display() → 30 20 10
pop → removes 30, top = 1 [10, 20]
display() → 10 20

2) Stack using STL:
→
```
#include <iostream>
#include <stack>
using namespace std;
int main(){
    stack<int> s;
    s.push(10);
    s.push(20);
    s.push(30);
    cout<<"Top: "<<s.top()<<endl;
        s.pop();
    cout<<"Stack elements:";
    while(!s.empty()){
        cout<<s.top()<<" ";
        s.pop();
    }
}
```

* O/P: →
Top = 30
Stack element: 20 10
=> Dry Run: →
Start s = [] (empty)
push(10) → [10], push(20) → [10, 20]
push(30) → [10, 20, 30]
top → 30
pop() → removes 30 → [10, 20]
While loop: print 20, pop → [10]
print 10, pop → []

3) Stack using linked list:
→
```
#include <iostream>
using namespace std;
struct Node{
    int data;
    Node* next;
};
    Node* top = NULL;
void push(int x){
    Node* temp = new Node();
    temp → data = x;
    temp → next = top;
    top = temp;
}
void pop(){
    if(top == NULL){
        cout<<"Underflow";
        return;
    }
```

```
cout<<top -> data <<" popped";
Node * temp = top;
top = top -> next;
delete temp;
}
cout << endl;
}
int main(){
    push(10);
    push(20);
    push(30);
    display();
    pop();
    display();
}
```

* O/P :→

```
30 20 10
30 popped
20 10
```

=> Dry Run: →

Start: top == NULL

push(10) → 10 → NULL (top = 10)

push(20) → 20 → 10 → NULL (top = 20)

push(30) → 30 → 20 → 10 → NULL (top = 30)

display() → 30 20 10

pop() → removes 30 → 20 → 10 → NULL
(top = 20)

display() → 20 10

4) Paranthesis Checking:-

→

```
#include <iostream>
#include<stack>
using namespace std;
int main() {
    string s;
    cin >> s;
    stack <char> st;
    for(char c:s){
  if(c=='('||<=='{'||c=='[') st.push(c);
    else {
  if(st.empty()){ cout<<" Not ~~Balanced~~ Around";
    return 0;
        }
  char t =st.pop(); st.pop();
  if ((c==')'&&t!='(')||(c=='}'&&t1='{')
  ||(c==']'&&t!='[')){
   cout<<"Not balanced";
  }
}
cout<<"(st.empty()?"Balanced":"Not Balanced");
}
```

* O/P :→

```
{[()]}
Balanced.
```

P.T.O.

5) Postfix evaluation:-

→
```
#include <iostream>
#include <stack>
using namespace std;
int evaluatePostfix (string exp){
    char c = exp[i];
if (isdigits (c)){
   st.push (c-'0');
   }
else {
   int b =st.top(); st.pop();
   int a =st.top(); st.pop();
   switch (c) {
   case'+': st.push (a+b); break;
   case'-': st.push (a-b); break;
   case'*': st.push (a*b); break;
   case'/': st.push (a/b); break;
    }
  }
}
return st.top(); }
Int main(){
    string exp ="23*54*+9-";
    cout<<"Result ="<<evaluatePostfix(exp);
  }
```

* O/P:→
17

=> Dry Run:→
23* → 6
54* → 20
6+20 → 26
26-9 → 17

25/4/26

Experiment No:10

1) Implementation of queue using Array:

→
```
#include<iostream>
using namespace std;
#Define MAX 5
int q[MAX], front = -1, rear = -1;
void enqueue(int x){
      if (rear == MAX-1){
         cout<<" overflow";
         return;
      }
      if (front == -1) front = 0;
      q[++rear] =x;
      }
      void dequeue(){
   if(front == -1) || (front >rear) {
      cout<<"Underflow";
      return;
      }
   cout<<"Deleted:"<<q[front ++];
      }
      void display(){
   if (front == -1) || (front >rear){
   cout<<" empty"; return;
      }
   for (int i = front; i<=rear; i++) {
         cout<< q[i] <<" ";
         cout<<endl;
      }
int main(){
      enqueue(10);
```

```
    enqueue(20);
    enqueue(30);
    display();
    dequeue();
    display();
}
```

* O/P: →
10 20 30
Deleted is 10
20 30

=> Dry Run: →
enqueue → 10 enqueue → 20 enqueue → 30
display → 10 20 30
dequeue → 10
display → 20 30

2) Implementation of Queue using STL:

→
```
#include<iostream>
#include<queue>
using namespace std;
int main(){
    queue<int> q;
    q.push(10);
    q.push(20);
    q.push(30);
cout<<"Front:"<<q.front()<<endl;
    q.pop();  while(!q.empty()){
                cout<<q.front()<<" "; q.pop();
  }
}
```

* O/P: → Front: 10
20 30

=> Dry Run: →
q = [ ]
push → [10, 20, 30]
front → 10 pop [20, 30]
print → 20 30

3) Implementation of Double Ended Queue using STL:

→
```
#include<iostream>
#include<dequeue>
using namespace std;
int main(){
    dequeue<int> dq;
    dq.push(10);
    dq.push(5);
    dq.push(20);
cout<<"Front:"<<dq.front()<<endl;
cout<<"Back:"<<dq.back()<<endl;
dq.pop_front();
dq.pop_back();
for(int x: dq) cout<<x<<" "; }
```

* O/P: → Front: 5
Back: 20
10

=> Dry Run: →
dq = [ ]
push_back(10) → [10]; push_front(5) → [5, 10]
push_back(20) → [5, 10, 20]
front → 5
back → 20
pop_front() → [10, 20]; pop_back() → [10]
print → 10

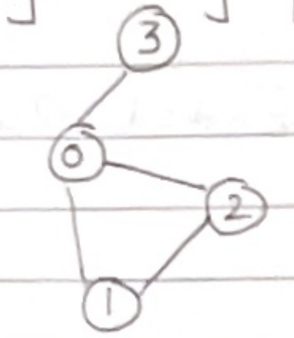
P.T.O.

25/4/26

Experiment No 12

Q1) Write adjacency matrix & adjacency list for below attached graph:

(i) For given graph find the adjacency matrix:



edges present: (0-1), (0-2), (0,-3), (1-2)

→ Adjacency Matrix:

	0	1	2	3
0	0	1	1	1
1	1	0	1	0
2	1	1	0	0
3	1	0	0	0

(ii) Adjacency List:

1) 0: 1, 2, 3
2) 1: 0, 2
3) 2: 0, 1
4) 3: 0

2) WAP for implementation of adjacency Matrix using STL:

→
```
#include <iostream>
#include <vector>
using namespace std;
int main(){
int n, edges;
cout << "Enter number of vertices:";
cin>>n;
vector <vector <int>> adj(n, vector <int>(n,0));
cout<<" Enter no. of edges";
cin >> edges;
cout<<" Enter edges:(u,v)";
for(int i=0; i<edges; i++){
    int u,v;
    cin >> u >> v;
    adj[u][v] = 1;
    adj[v][u] = 1; }
cout <<" Adjacency matrix :";
for(int i=0; i<n; i++){
    for(int j=0; j<n; j++) {
        cout << adj[i][j] <<" ";
    }
cout << endl; } }
```

* O/P: →

```
Enter no. of vertices : 4
Enter no. of edges : 4
Enter edges:
0 1
0 2
0 3
0 4
1 2
```